

THE HARTFORD- HCL GUVI: TRAINING PROGRAM FOR .NET FULL STACK (Including Conceptual Overview of advanced Topics)

Program Schedule Model:

Audience: Tier-1 College Graduate Developers & Engineers

Duration : 14 Weeks (11 weeks : Core skillset; 1 week: Capstone; 1 week: Softskills; 1 week: GenAI) : 8 Hours per day

Delivery Mode: Theory + Live Demos + Hands-on Practice

- Instructor: Industry Mentors
- Session-wise Gamified Quiz challenges (for better interactions & knowledge sharing)
- Hands-on practice with use cases

Learning Methodology

Methodology	Percentage
Instructor-Led Theory & Demonstration	15%
Guided Hands-on Labs	60%

Project-Based Learning	25%
------------------------	-----

Topics with a high level Conceptual Overview Only:

- Github Actions, Github Workflows, Container image build
- Jenkins Pipelines, Pipeline as Code, Nexus Repository
- SQL Troubleshooting, No SQL Databases, SQL Cursors, Triggers, Database Debugging Techniques
- Concurrency
- Event Streaming
- Introduction to MCP
- Angular Micro Frontend Architecture
- AWS Lambda, AWS ECR/ECS Overview, AWS SQS, Terraform, AWS Service Catalog, Cloud Formation

Assessments:

Component 1: Weekly assessments (MCQ + Coding assessment / Project-based assessment)
to be conducted every week

Component 2 : Checkpoint assessments (After Milestones)

Checkpoint	Timeline	Coverage
Checkpoint 1: Programming Foundations	End of Week 2	C#, OOP, LINQ, ASP.NET Core APIs

Checkpoint 2: Backend Development	End of Week 4	EF Core, Database Integration, Authentication, API Security
Checkpoint 3: Enterprise Backend Engineering	End of Week 6	Clean Architecture, Logging, Monitoring, Microservices, Docker
Checkpoint 4: Frontend Development	End of Week 8	HTML, CSS, JavaScript, TypeScript, Angular
Checkpoint 5: Full Stack, DevOps & Cloud	End of Week 11	Full Stack Integration, CI/CD, AWS, Agile
Final Capstone Assessment	End of Week 14	End-to-End Enterprise Full Stack Application

Approach to Hands-on Learning and Capstone Execution

Component	Approach
Instructor-Led Concept Sessions	Key concepts are introduced through interactive lectures, demonstrations, and real-world industry examples to establish foundational understanding.
Guided Hands-on Labs	Every topic is followed by structured lab exercises where learners implement concepts under instructor guidance using industry-standard tools and frameworks.

Practice Assignments	Weekly assignments reinforce learning through individual coding, database, frontend, backend, or automation tasks.
Scenario-Based Learning	Learners work on business scenarios and case studies that simulate real-world enterprise challenges and workflows.
Code Reviews & Feedback	Regular mentor reviews help learners improve coding standards, design practices, debugging techniques, and solution quality.
Capstone Project Execution	Learners develop a comprehensive end-to-end project that integrates frontend, backend, database, cloud, DevOps, AI, and automation testing components.
Project Mentoring	Dedicated mentor checkpoints are conducted for requirement analysis, architecture review, implementation guidance, testing, deployment, and project refinement.
Project Demonstration & Evaluation	Learners present their solution through live demonstrations, technical walkthroughs, code reviews, and project documentation reviews.

Continuous Assessment Approach

Assessment Component	Frequency	Purpose
Knowledge Checks & Quizzes	Daily	Validate conceptual understanding of topics covered during the week
Hands-on Exercises	Daily	Assess practical implementation skills and tool proficiency
Assignments	Bi-Weekly	Evaluate problem-solving ability and coding standards
Code Reviews	After checkpoint assessment	Improve code quality, maintainability, and adherence to best practices
Readiness Reviews	Final Phase	Assess employability and project articulation capabilities

Program Objectives

- Build strong foundational understanding of core concepts covered in the program
- Enable learners to apply concepts through hands-on practice and real-world scenarios
- Develop problem-solving and analytical thinking skills
- Strengthen practical implementation using tools and frameworks included in the curriculum
- Prepare learners for project-based execution and industry-relevant use cases
- Improve collaboration, communication, and structured thinking
- Introduce AI awareness and productivity enhancement techniques
- Prepare learners for workplace readiness through soft skills and capstone execution

Capstone Evaluation Criteria

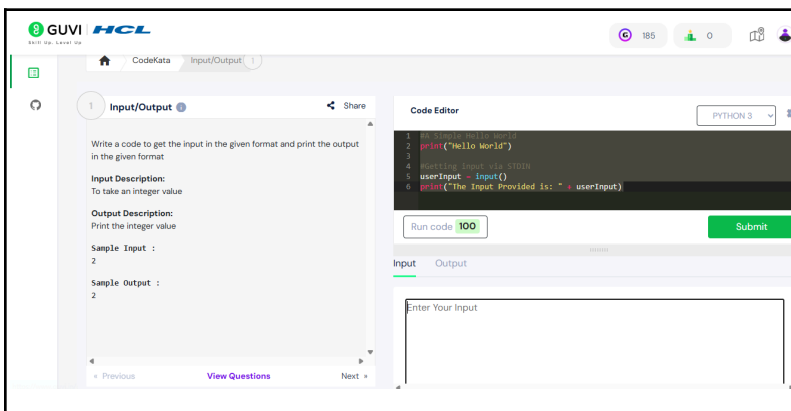
Evaluation Area	Weightage
Frontend Design & User Experience	10%
Backend Development & Business Logic	15%
Database Design & Data Management	10%
Functional Completeness & Requirement Coverage	15%
Code Quality & Best Practices	10%
Testing & Quality Assurance	10%
Deployment, DevOps & CI/CD Implementation	10%
Documentation Quality	5%
Project Demonstration & Presentation	10%
Innovation, Problem Solving & Technical Decision Making	5%

Readiness Benchmarks for Deployment

Readiness Area	Benchmark
Attendance	Minimum 80% program attendance
Technical Assessments	Minimum 60% average score across checkpoints

Lab Completion	Completion of all mandatory hands-on exercises
Assignment Submission	Submission of all major assignments and mini projects
Capstone Project	Successful completion and demonstration of capstone project
Presentation Skills	Ability to explain architecture, implementation, and business value

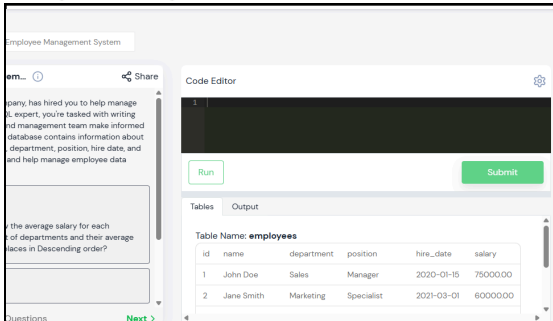
GUVI's Exclusive Practice Platforms:



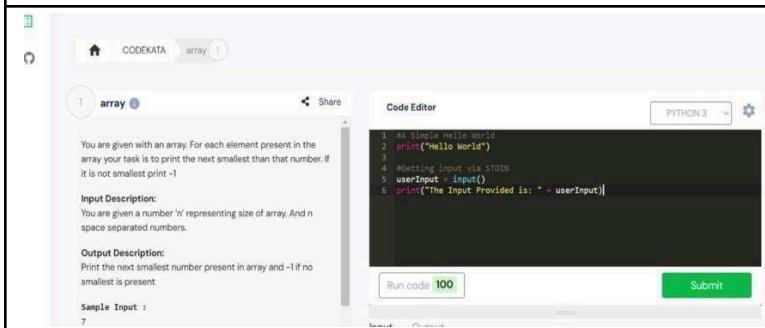
CodeKata : <https://www.guvi.in/code-kata/>
A comprehensive toolkit designed to enhance coding skills and thoroughly prepare you for success in technical interviews.



Webkata: <https://www.guvi.in/webkata.html>
A cloud-based module designed to refine your front-end skills, eliminating the need for complex local environment setup.



SQLKata: <https://www.guvi.in/sqlkata/sql/>
an interactive environment that enables you to write, execute, and test SQL queries, helping you sharpen your database skills without the need for complex setups.



IDE : <https://www.guvi.in/ide/>
GUVI IDE is an Integrated Development Environment (IDE) that allows you to write, edit, run, test, and debug your code seamlessly in one platform.

Week-wise Program Summary:

Week	Module	Key Topics
Week 1	C# & .NET Fundamentals	.NET Ecosystem, C# Fundamentals, OOP Concepts, Collections, Exception Handling, LINQ, Delegates, Lambda Expressions, Async Programming
Week 2	ASP.NET Core API Development	ASP.NET Core Architecture, REST APIs, Controllers, Routing, Middleware, Dependency Injection, Configuration Management, Swagger/OpenAPI

Week 3	Backend Development with SQL Server & Entity Framework Core	Database Fundamentals, SQL Server, MongoDB basics, CRUD Operations, Advanced SQL, Stored Procedures, Entity Framework Core, Migrations, Repository Pattern, Layered Architecture
Week 4	Secure API Development & Authentication	Authentication & Authorization, JWT, Role-Based Access Control, Claims-Based Security, API Security Best Practices, Secure Coding, Exception Handling, API Testing with Playwright
Week 5	Advanced Backend Development & Enterprise Architecture	SOLID Principles, Design Patterns, Clean Architecture, Layered Architecture, Advanced EF Core, Query Optimization, Transactions, Logging (Serilog), Global Exception Handling, Caching, Performance Optimization
Week 6	Microservices Architecture & Backend Integration	Monolithic vs Microservices, Domain-Driven Design Concepts, Service Design, API Versioning, Data Management Strategies, Integration Patterns, Event-Driven Architecture Concepts, API Gateway, Service Discovery Concepts
Week 7	Frontend Foundations	HTML5, CSS3, Responsive Design, Flexbox, CSS Grid, Tailwind CSS, UI Development Best Practices
Week 8	JavaScript, TypeScript & Angular Development	JavaScript ES6+, TypeScript, Angular Architecture, Components, Services, Dependency Injection, Forms, Routing, State Management, API Consumption
Week 9	Full Stack Integration & System Hardening	Angular–ASP.NET Core Integration, Authentication Integration, Role-Based UI Security, State Management, Error Handling, Logging Integration, Performance Optimization, Security Hardening, Production Readiness

Week 10	DevOps & CI/CD	DevOps Fundamentals, Git, GitHub, Branching Strategies, Pull Requests, Docker, Containerization, GitHub Actions, CI/CD Pipelines, Release Management, Deployment Strategies
Week 11	AWS Cloud & Agile Methodologies	Agile Principles, Scrum Framework, AWS Fundamentals, EC2, S3, IAM, RDS, VPC Basics, Application Deployment on AWS, CloudWatch Monitoring, Cloud Operations & Cost Optimization
Week 12	Soft Skills	Communication skills, teamwork, interview preparation, resume building, presentation skills
Week 13	AI Integration	Introduction to AI tools, AI-assisted development, integrating AI features into applications
Week 14	Capstone Project	End-to-end fullstack project implementation covering frontend, backend, database, and deployment

Detailed Weekly Curriculum

Week 1 – C# & .NET Fundamentals

Day 1 – Introduction to .NET Platform & C# Basics

Module 1.1: Introduction to .NET Platform

- Overview of .NET Ecosystem
- CLR, CTS and CLS
- Managed vs Unmanaged Code

- Development Environment Setup

Module 1.2: C# Language Fundamentals

- Program Structure
- Variables & Constants
- Data Types
- Operators

Practice Session 1

- Create Console Application
- User Input & Output Programs

Module 1.3: String Handling & Type Conversion

- String Operations
- StringBuilder
- Type Casting

Practice Session 2

- Employee Registration Console Application

Daily Deliverable

- Working Console Application
-

Day 2 – Control Flow & Methods

Module 2.1: Decision Making

- If Else Statements
- Switch Statements

Module 2.2: Loops & Iterations

- For Loop
- While Loop
- Do While Loop

Practice Session 1

- Insurance Premium Calculator

Module 2.3: Methods & Reusability

- Method Creation
- Parameters & Return Types
- Method Overloading

Practice Session 2

- Refactor Logic into Methods

Daily Deliverable

- Modular Console Application
-

Day 3 – Object Oriented Programming

Module 3.1: Classes & Objects

- Classes
- Objects
- Constructors

Module 3.2: Core OOP Concepts

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Practice Session 1

- Customer Management System

Module 3.3: Interfaces & Abstract Classes

- Interface Design
- Abstract Classes

Practice Session 2

- OOP Refactoring Exercise

Daily Deliverable

- OOP Based Application
-

Day 4 – Collections & Exception Handling

Module 4.1: Collections

- Arrays
- Lists
- Dictionaries
- HashSets

Module 4.2: Collection Processing

- Searching
- Filtering
- Sorting

Practice Session 1

- Employee Records Management

Module 4.3: Exception Handling

- Try-Catch
- Finally
- Custom Exceptions

Practice Session 2

- Error Handling Implementation

Daily Deliverable

- Collection Based Application
-

Day 5 – LINQ, Delegates & Async Programming

Module 5.1: LINQ Fundamentals

- Query Syntax
- Method Syntax
- Filtering & Grouping

Module 5.2: Delegates & Lambda Expressions

- Delegates
- Multicast Delegates

- Lambda Expressions

Practice Session 1

- Employee Analytics Application

Module 5.3: Asynchronous Programming

- Task
- Async & Await
- Parallel Execution Concepts

Practice Session 2

- Async Data Processing

Daily Deliverable

- Enterprise Console Application using LINQ & Async Programming

Week 2 – ASP.NET Core API Development

Day 1 – Introduction to ASP.NET Core & Web APIs

Module 1.1: ASP.NET Core Fundamentals

- Introduction to ASP.NET Core
- ASP.NET Core Architecture
- Web Applications vs Web APIs
- Request Processing Pipeline

Module 1.2: Creating ASP.NET Core Web APIs

- Project Templates
- Solution Structure
- appsettings.json
- launchSettings.json

Practice Session 1

- Create First ASP.NET Core Web API

Module 1.3: Controllers & Routing

- Controllers
- Action Methods
- Attribute Routing

- Route Parameters

Practice Session 2

- Develop Employee Management API

Daily Deliverable

- Functional REST API with Controllers & Routing
-

Day 2 – HTTP Methods & API Development

Module 2.1: RESTful API Principles

- REST Architecture
- Resources & Endpoints
- HTTP Verbs
- Status Codes

Module 2.2: CRUD Operations

- GET
- POST
- PUT
- DELETE

Practice Session 1

- Product Management API

Module 2.3: Model Binding & Validation

- Request Models
- Data Annotations
- Model Validation
- Custom Validation

Practice Session 2

- Implement Request Validation

Daily Deliverable

- CRUD-Based REST API
-

Day 3 – Middleware & Dependency Injection

Module 3.1: Middleware Fundamentals

- Middleware Pipeline
- Built-in Middleware
- Request Processing Flow

Module 3.2: Custom Middleware

- Creating Middleware
- Request Logging
- Exception Handling Middleware

Practice Session 1

- Develop Logging Middleware

Module 3.3: Dependency Injection

- DI Concepts
- Service Lifetimes
- Registering Services
- Concurrency - high level conceptual overview

Practice Session 2

- Implement Service-Based Architecture

Daily Deliverable

- API with Custom Middleware & DI
-

Day 4 – API Documentation & Configuration

Module 4.1: Configuration Management

- Configuration Sources
- Environment Variables
- appsettings Files

Module 4.2: Swagger/OpenAPI

- Swagger Setup
- API Documentation

- API Testing through Swagger

Practice Session 1

- Generate Interactive API Documentation

Module 4.3: Logging Fundamentals

- ILogger
- Logging Levels
- Logging Best Practices

Practice Session 2

- Add Logging to APIs

Daily Deliverable

- Documented API with Logging
-

Day 5 – Mini Project

Module 5.1: Error Handling

- Global Exception Handling
- Standard Response Models

Practice Session 1

- Test API Endpoints using Postman

Module 5.2: Mini Project

- Employee/Customer Management API
- CRUD Operations
- Validation
- Logging

Practice Session 2

- Project Completion & Review

Daily Deliverable

- RESTful API Project

Week 3 – Backend Development with SQL Server & Entity Framework Core

Day 1 –SQLServer Fundamentals

Module 1.1: Database Fundamentals

- Relational Databases
- Database Design Concepts
- Tables, Keys & Relationships

Module 1.2: SQL Server Basics

- SQL Server Architecture
- Creating Databases
- Creating Tables

Practice Session 1

- Design Employee Database

Module 1.3: SQL CRUD Operations

- Insert
- Update
- Delete
- Select

Practice Session 2

- Employee Database Operations

Daily Deliverable

- Normalized Database Design
-

Day 2 – Advanced SQL Development

Module 2.1: Querying Data

- Filtering
- Sorting
- Aggregations

Module 2.2: SQL Joins

- Inner Join
- Left Join
- Right Join
- Full Join

Practice Session 1

- Customer-Order Reporting Queries

Module 2.3: Advanced SQL Objects

- Views
- SQL Cursors - high level conceptual overview
- Stored Procedures- high level conceptual overview
- Functions- high level conceptual overview
- Triggers - high level conceptual overview
- SQL troubleshooting - high level conceptual overview
- NoSQL basics
- Database debugging techniques - high level conceptual overview

Practice Session 2

- Create Business Reports

Daily Deliverable

- Reporting Queries & Procedures
-

Day 3 – Introduction to Entity Framework Core

Module 3.1: ORM Fundamentals

- What is ORM?
- Benefits of EF Core
- EF Core Architecture

Module 3.2: DbContext & DbSet

- Entity Configuration
- Database Connections
- Context Management

Practice Session 1

- Create EF Core Project

Module 3.3: Code First Approach

- Entity Classes
- Migrations
- Database Generation

Practice Session 2

- Build Database using Migrations

Daily Deliverable

- EF Core Application
-

Day 4 – Data Access Layer Development

Module 4.1: CRUD Operations with EF Core

- Add Records
- Update Records
- Delete Records
- Retrieve Records

Module 4.2: LINQ with EF Core

- Filtering
- Sorting
- Projections
- Aggregations

Practice Session 1

- Employee Repository Development

Module 4.3: Repository Pattern

- Repository Design
- Generic Repository
- Benefits & Best Practices

Practice Session 2

- Implement Repository Layer

Daily Deliverable

- Repository-Based Data Access Layer
-

Day 5 – API & Database Integration

Module 5.1: Connecting APIs to Databases

- Database Connectivity
- Service Layer Design

Module 5.2: Layered Architecture

- Controller Layer
- Service Layer
- Repository Layer

Practice Session 1

- Build End-to-End Backend Flow

Module 5.3: Mini Project

- Employee Management Backend
- API + SQL Server + EF Core Integration

Practice Session 2

- Project Review & Optimization

Daily Deliverable

- Database-Driven ASP.NET Core Application
-

Week 4 – Secure API Development & Authentication

Day 1 – Authentication Fundamentals

Module 1.1: Security Fundamentals

- Authentication vs Authorization
- Common Security Threats
- OWASP Overview
- API Security Principles

Module 1.2: Identity & Access Management

- User Identity Concepts
- Claims-Based Authentication
- Role-Based Security

Practice Session 1

- Design Secure User Authentication Flow

Module 1.3: Introduction to JWT

- JWT Structure
- Access Tokens
- Token Lifecycle

Practice Session 2

- Generate and Validate JWT Tokens

Daily Deliverable

- JWT-enabled Authentication Module
-

Day 2 – Implementing JWT Authentication

Module 2.1: JWT Configuration

- JWT Middleware
- Token Generation
- Secret Key Management

Module 2.2: Securing API Endpoints

- Protected Routes
- Authorize Attribute
- Authentication Middleware

Practice Session 1

- Secure Existing API Endpoints

Module 2.3: Refresh Tokens

- Refresh Token Concepts
- Token Expiration
- Session Management

Practice Session 2

- Implement Refresh Token Mechanism

Daily Deliverable

- Secure Authentication-enabled API
-

Day 3 – Authorization & Role Management

Module 3.1: Authorization Fundamentals

- Role-Based Access Control (RBAC)
- Policy-Based Authorization
- Claims-Based Authorization

Module 3.2: Role Management

- User Roles
- Access Permissions
- Resource Protection

Practice Session 1

- Implement Admin/User Roles

Module 3.3: Secure API Design

- Endpoint Security
- Access Restrictions
- Security Best Practices

Practice Session 2

- Secure Business APIs

Daily Deliverable

- Role-Based Secure API
-

Day 4 – API Testing with Playwright & Security Best Practices

Module 4.1: API Security Testing

- Postman Authentication Testing
- Token Validation Testing
- Security Test Scenarios
- Playwright for API testing
- Introduction to Model Context Protocol - conceptual overview

Module 4.2: Secure Coding Practices

- Input Validation
- Data Sanitization
- Secure Configuration

Practice Session 1

- Security Testing Exercises

Module 4.3: Exception Handling & Error Responses

- Secure Error Handling
- Standardized Responses
- Error Logging

Practice Session 2

- Implement Global Exception Handling

Daily Deliverable

- Security-Tested API
-

Day 5 – Authentication & Authorization Project

Module 5.1: Secure Backend Design

- Authentication Flow Design
- Authorization Matrix

Module 5.2: Mini Project Development

- User Registration
- Login
- Role Management
- Protected APIs

Practice Session 1

- End-to-End Security Implementation

Module 5.3: Project Review & Optimization

- Security Review
- Best Practices Validation

Practice Session 2

- Code Refactoring

Daily Deliverable

- Secure Employee Management Backend
-

Week 5 – Advanced Backend Development & Enterprise Architecture

Day 1 – SOLID Principles & Clean Coding

Module 1.1: Clean Code Fundamentals

- Readability & Maintainability
- Coding Standards
- Refactoring Techniques

Module 1.2: SOLID Principles

- Single Responsibility Principle
- Open Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Practice Session 1

- Refactor Existing API using SOLID Principles

Module 1.3: Design Patterns Overview

- Repository Pattern
- Service Layer Pattern
- Factory Pattern

Practice Session 2

- Apply Design Patterns to Existing Application

Daily Deliverable

- Refactored Enterprise API
-

Day 2 – Layered & Clean Architecture

Module 2.1: Application Architecture

- Monolithic Architecture
- Layered Architecture
- Enterprise Application Structure

Module 2.2: Clean Architecture

- Domain Layer
- Application Layer
- Infrastructure Layer
- Presentation Layer

Practice Session 1

- Design Backend Architecture

Module 2.3: Dependency Management

- Separation of Concerns
- Dependency Flow
- Maintainability

Practice Session 2

- Convert Existing Application into Clean Architecture

Daily Deliverable

- Clean Architecture-Based Solution
-

Day 3 – Advanced Entity Framework Core

Module 3.1: Advanced EF Core

- Relationships
- Lazy Loading
- Eager Loading
- Explicit Loading

Module 3.2: Query Optimization

- LINQ Optimization
- Tracking vs No Tracking
- Performance Considerations

Practice Session 1

- Optimize Database Queries

Module 3.3: Transactions & Concurrency

- Transactions
- Concurrency Handling
- Unit of Work Concepts

Module 3.4 : Event Streaming - High Level conceptual overview

Practice Session 2

- Implement Transaction Management

Daily Deliverable

- Optimized Data Access Layer
-

Day 4 – Logging & Exception Management

Module 4.1: Logging Fundamentals

- Logging Concepts
- Log Levels
- Structured Logging

Module 4.2: Serilog

- Configuration
- File Logging
- Application Logging

Practice Session 1

- Integrate Logging Framework

Module 4.3: Global Exception Handling

- Middleware-Based Exception Handling
- Error Response Standardization
- Error Tracking

Module 4.4 Unit Testing with NUnit

- Test Fundamentals
- NUnit Framework
- Test Fixtures
- Assertions
- Mocking
- API & Service Layer Testing

Practice Session 2

- Implement Global Exception Framework

Daily Deliverable

- Production-grade Logging & Error Handling
-

Day 5 – Performance Optimization & Backend Assessment

Module 5.1: API Performance

- Performance Bottlenecks
- Response Optimization
- API Best Practices

Module 5.2: Caching Concepts

- In-Memory Caching
- Distributed Cache Overview

Practice Session 1

- Improve Existing APIs

Module 5.3: Backend Assessment

- Architecture Review
- Code Review
- Performance Review

Practice Session 2

- Backend Enhancement Activity

Daily Deliverable

- Optimized Enterprise Backend
-

Week 6 – Microservices Architecture & Backend Integration

Day 1 – Microservices Fundamentals

Module 1.1: Enterprise Architecture Evolution

- Monolithic Architecture
- Service-Oriented Architecture
- Microservices Architecture

Module 1.2: Microservices Principles

- Service Boundaries
- Loose Coupling
- High Cohesion

Practice Session 1

- Identify Service Boundaries

Module 1.3: Domain Driven Design Concepts

- Bounded Context
- Aggregates
- Domain Modeling

Practice Session 2

- Design Domain Models

Daily Deliverable

- Microservices Architecture Design
-

Day 2 – Designing Microservices

Module 2.1: Service Design

- API Design Standards
- Resource Modeling
- Versioning Strategies

Module 2.2: Data Management Strategies

- Database per Service
- Shared Database Challenges
- Data Consistency

Practice Session 1

- Design Multiple Services

Module 2.3: Service Governance

- API Standards
- Service Contracts
- Documentation

Practice Session 2

- Create Service Specifications

Daily Deliverable

- Microservice Design Specifications
-

Day 3 – Backend Integration Patterns

Module 3.1: Integration Fundamentals

- System Integration Concepts
- Point-to-Point Integration
- Enterprise Integration Patterns

Module 3.2: API-Based Integration

- RESTful Service Communication
- API Contracts
- Service Dependencies

Practice Session 1

- Integrate Multiple APIs

Module 3.3: Event-Driven Architecture Concepts

- Event Producers
- Event Consumers
- Asynchronous Communication Concepts

Practice Session 2

- Design Event-Based Workflows

Daily Deliverable

- Integrated Backend Services
-

Day 4 – Enterprise API Management

Module 4.1: API Gateway Concepts

- API Gateway Architecture
- Routing
- Aggregation

Module 4.2: Service Discovery Concepts

- Dynamic Discovery
- Registry Concepts
- Scalability Considerations

Practice Session 1

- API Gateway Design Exercise

Module 4.3: Security in Distributed Systems

- Service Authentication
- Secure Communication
- Distributed Authorization

Practice Session 2

- Secure Multi-Service Architecture

Daily Deliverable

- Enterprise Integration Architecture
-

Day 5 – Backend Capstone Project

Module 5.1: Enterprise Backend Solution

- Multi-Service Architecture Design
- API Integration
- Security Integration

Module 5.2: Backend Implementation

- Service Development
- Integration Development

Practice Session 1

- Build End-to-End Backend Solution

Module 5.3: Architecture Review

- Design Review

- Code Review
- Optimization Recommendations

Practice Session 2

- Project Demonstration

Daily Deliverable

- Enterprise Backend Application with Integrated Services
-

WEEK 7: Angular Foundation + Core Development

DAY 1: HTML + CSS Refresher (Enterprise UI Focus)

MODULE 1.1: Semantic HTML for Application UI

- semantic structure for enterprise apps
- form layouts and UI structure standards
- accessibility basics (ARIA overview)

Practice Session:

Design login + dashboard UI structure for e-commerce system

MODULE 1.2: CSS Layout Systems

- flexbox layout system
- grid layout system
- responsive design patterns

Practice Session:

Build responsive product listing page layout

MODULE 1.3: UI Component Thinking

- UI decomposition strategy
- reusable UI blocks
- layout hierarchy design

Practice Session:

Break homepage into reusable components

MODULE 1.4: UI Flow Mapping

- UI → API mapping concept
- page navigation flow design
- frontend structure planning

Practice Session:

Design full user journey (login → product → cart)

DAILY DELIVERABLE

- responsive UI layout
 - UI architecture breakdown
-

DAY 2: JavaScript + TypeScript Refresher (Application Grade)

MODULE 2.1: JavaScript for Applications

- objects, arrays, functions
- async/await patterns
- event-driven programming

Practice Session:

Build product filtering logic

MODULE 2.2: TypeScript Core

- interfaces and typing system
- class-based modeling
- API response typing

Practice Session:

Convert JS model to TypeScript

MODULE 2.3: API Consumption Basics

- fetch/HttpClient concept
- JSON handling
- async data flow

Practice Session:

Simulate product API integration

MODULE 2.4: UI State Basics

- state representation

- cart logic design
- UI update flow

Practice Session:

Build cart state logic

DAILY DELIVERABLE

- typed frontend logic system
 - API-ready frontend foundation
-

DAY 3: Angular Setup + Components + Templates + Data Binding

MODULE 3.1: Angular Project Setup

- Angular CLI setup
- project structure
- environment configuration

Practice Session:

Initialize Angular project and structure folders

MODULE 3.2: Angular Components

- component architecture
- lifecycle basics
- reusable components

Practice Session:

Create product and header components

MODULE 3.3: Angular Templates

- interpolation
- structural templates
- binding syntax

Practice Session:

Build product UI template

MODULE 3.4: Data Binding

- one-way binding
- property binding
- event binding
- two-way binding

Practice Session:

Build interactive product input UI

DAILY DELIVERABLE

- Angular project setup
 - working components + templates + binding
-

DAY 4: Services + HTTP Client Integration

MODULE 4.1: Angular Services

- service architecture
- dependency injection
- separation of concerns

Practice Session:

Create product service

MODULE 4.2: HttpClient Integration

- GET/POST/PUT/DELETE
- observable handling
- async flow

Practice Session:

Fetch product data from backend

MODULE 4.3: Async Data Handling

- subscription model
- error handling
- response mapping

Practice Session:

Render API-driven product list

MODULE 4.4: API Architecture in Angular

- service-based API abstraction

- frontend-backend flow design

Practice Session:

Design API layer for e-commerce UI

DAILY DELIVERABLE

- API-integrated Angular service layer
 - dynamic UI rendering
-

DAY 5: Reactive Forms

MODULE 5.1: Reactive Forms Core

- FormGroup and FormControl
- form structure design
- reactive form lifecycle

Practice Session:

Build registration form

MODULE 5.2: Validation System

- built-in validators
- custom validation
- error handling UI

Practice Session:

MODULE 5.3: Form Submission Flow

- API integration with forms
- success/failure handling

Practice Session:

Connect form to backend API

MODULE 5.4: Form UX Design

- error states
- disabled states
- user feedback design

Practice Session:

Improve form UX behavior

WEEK 7 DELIVERABLE

- Angular foundation system
 - API-connected UI module
 - reactive forms implementation
-

WEEK 8: Advanced Angular + Architecture + Testing Awareness

DAY 1: Standalone Components + Modern Angular Structure

MODULE 1.1: Standalone Components

- standalone architecture
- module-free design
- modern Angular approach

Practice Session:

Convert existing components to standalone

MODULE 1.2: Angular Architecture Design

- feature-based structure
- scalable folder design
- separation of concerns

Practice Session:

Restructure project into feature modules

MODULE 1.3: Component Reusability

- reusable UI components
- shared components design

Practice Session:

Build reusable product card component

MODULE 1.4: Service Optimization

- shared services
- API abstraction refinement

Practice Session:

Optimize product service layer

DAILY DELIVERABLE

- modern Angular structure
 - reusable component system
-

DAY 2: Component Communication + State Management

MODULE 2.1: @Input / @Output Communication

- parent-child communication
- event flow design

Practice Session:

Build cart interaction system

MODULE 2.2: Service-Based State

- shared state design
- centralized data flow

Practice Session:

MODULE 2.3: Angular Signals

- reactive state system
- modern Angular reactivity

Practice Session:

Convert cart state to signals

MODULE 2.4: UI State Architecture

- local vs global state
- UI state design patterns

Practice Session:

Design state model for e-commerce app

DAILY DELIVERABLE

- working state management system
 - component communication model
-

DAY 3: Routing + Lazy Loading + Navigation

MODULE 3.1: Angular Routing

- route configuration
- navigation flow
- route parameters

Practice Session:

Implement login → product → cart routing

MODULE 3.2: Lazy Loading

- feature module loading
- performance optimization

Practice Session:

Lazy load product module

MODULE 3.3: Route Guards

- authentication protection
- route security

Practice Session:

Protect cart route

MODULE 3.4: Navigation UX Flow

- user journey design
- navigation optimization

Practice Session:

Design full app navigation flow

DAILY DELIVERABLE

- routing system with lazy loading
 - protected navigation flow
-

DAY 4: Directives + Pipes + UI Optimization

MODULE 4.1: Angular Directives

- structural directives
- attribute directives

Practice Session:

Build conditional product rendering

MODULE 4.2: Angular Pipes

- built-in pipes
- custom pipes

Practice Session:

Build price formatting pipe

MODULE 4.3: UI Optimization

- rendering performance
- change detection basics

Practice Session:

MODULE 4.4: UI Enhancement

- dynamic UI improvements
- UX refinements

Practice Session:

Improve cart UI experience

Module 4.5 : Angular Micro Frontend Architecture - High level Conceptual Overview

DAILY DELIVERABLE

- optimized Angular UI system
-

DAY 5: Unit Testing + UI Testing + Angular Project

MODULE 5.1: Unit Testing Basics

- Jasmine framework
- component testing
- service testing

Practice Session:

Test product service

MODULE 5.2: Component Testing

- DOM testing
- input/output validation

Practice Session:

Test product component

MODULE 5.3: UI Testing (Manual)

- user flow validation
- form testing
- navigation testing

Practice Session:

Test full Angular workflow

MODULE 5.4: Angular Project Integration

- full UI system integration
- frontend assembly

Practice Session:

Build mini e-commerce frontend

WEEK 8 DELIVERABLE

- complete Angular application
- state + routing + services + testing
- production-like frontend system

Week 9 – Full Stack Integration & System Hardening

Day 1 – Frontend & Backend Integration

Module 1.1: Full Stack Architecture Overview

- Frontend-Backend Communication
- API Consumption Patterns
- End-to-End Request Lifecycle

Module 1.2: Angular Service Integration

- HttpClient
- Service Layer Development
- API Consumption

Practice Session 1

- Connect Angular Application to ASP.NET Core APIs

Module 1.3: Data Flow Management

- Request Handling
- Response Processing
- Error Propagation

Practice Session 2

- Build Integrated CRUD Workflows

Daily Deliverable

- Fully Connected Angular & .NET Application
-

Day 2 – Authentication & Authorization Integration

Module 2.1: Authentication Flow

- Login Process
- JWT Token Flow
- Session Management

Module 2.2: Frontend Security

- Route Guards
- Secure Storage
- Token Management

Practice Session 1

- Secure Angular Application

Module 2.3: Authorization Implementation

- Role-Based Access
- Protected Components
- Dynamic Navigation

Practice Session 2

- Implement Multi-Role Access Control

Daily Deliverable

- Secure Full Stack Application
-

Day 3 – Advanced Integration & State Management

Module 3.1: Advanced API Integration

- Interceptors
- Error Handling
- Retry Mechanisms

Module 3.2: State Management Concepts

- Shared Services
- State Management Principles
- Application State Design

Practice Session 1

- Manage Application State

Module 3.3: Performance Optimization

- Lazy Loading
- API Optimization
- Client-Side Optimization

Practice Session 2

- Optimize Existing Application

Daily Deliverable

- Optimized Full Stack Solution
-

Day 4 – System Hardening & Production Readiness

Module 4.1: Security Hardening

- Secure Configuration Management
- Secrets Handling
- Input Validation Review

Module 4.2: Logging & Monitoring Integration

- Frontend Error Logging
- Backend Logging Review
- Monitoring Readiness

Practice Session 1

- Production Readiness Assessment

Module 4.3: Performance & Reliability

- Application Health Checks
- Resilience Concepts
- Scalability Considerations

Practice Session 2

- Hardening Existing Application

Daily Deliverable

- Production-Ready Full Stack Application
-

Day 5 – Enterprise Full Stack Project

Module 5.1: End-to-End Application Development

- User Management
- Business Workflow Integration
- Secure API Consumption

Module 5.2: System Validation

- Functional Testing
- Security Validation
- Performance Validation

Practice Session 1

- End-to-End Application Testing

Module 5.3: Project Review & Optimization

- Code Review
- Architecture Review
- Best Practices Assessment

Practice Session 2

- Final Project Demonstration

Daily Deliverable

- Enterprise Full Stack Application
-

Week 10 – DevOps & CI/CD

Day 1

- DevOps Fundamentals
- DevOps Lifecycle
- Jenkins Pipelines - High level conceptual overview
- Pipeline as Code - High level Conceptual overview
- Nexus Repository - High level conceptual overview
- Version Control with Git
- Git Branching Strategies

Day 2

- GitHub Workflows
- Pull Requests
- Code Reviews
- Repository Management
- Github Actions, Github Workflows - High level Conceptual overview
-

Day 3

- Introduction to Docker
- Containers vs Virtual Machines
- Docker Images & Containers
- Container Image Build - High level o
- Dockerfile Creation

Day 4

- CI/CD Concepts
- GitHub Actions
- Automated Build Pipelines
- Deployment Pipelines

Day 5

- Infrastructure as Code Concepts
- Release Management
- Deployment Strategies
- Blue-Green & Rolling Deployments

Deliverable

- Containerized Application with Automated CI/CD Pipeline
-

Week 11 – AWS Cloud & Agile Methodologies

Day 1 – Agile Fundamentals

Module 1.1: Agile Principles

- Agile Manifesto
- Agile Values
- Agile Mindset

Module 1.2: Scrum Framework

- Scrum Roles
- Scrum Ceremonies
- Scrum Artifacts

Practice Session

- Sprint Planning Simulation
-

Day 2 – AWS Fundamentals

Module 2.1: Cloud Computing Concepts

- Cloud Service Models
- AWS Global Infrastructure
- AWS Shared Responsibility Model

Module 2.2: Core AWS Services

- EC2
- S3
- IAM

Practice Session

- Launch and Configure EC2 Instance
-

Day 3 – AWS Data & Application Services

Module 3.1: Database Services

- Amazon RDS
- Database Connectivity

Module 3.2: Networking Basics

- VPC Concepts
- Security Groups
- Load Balancers
- subnets (public/private)
- security groups vs NACLs
- IaC - high level conceptual overview
 - Terraform - High level conceptual overview
 - AWS CloudFormation
- AWS Service Catalog - High level conceptual overview
- AWS Lambda basics - High level conceptual overview
- AWS SQS - High level conceptual overview
- AWS ECR & ECS - High level conceptual overview

Practice Session:

Design secure network architecture

Practice Session

- Configure AWS Resources
-

Day 4 – Application Deployment on AWS

Module 4.1: Deploying ASP.NET Core Applications

- Deployment Planning
- Application Hosting

Module 4.2: Frontend Deployment

- Static Website Hosting on S3
- Application Configuration

Practice Session

- Deploy Full Stack Application
-

Day 5 – Monitoring, Operations & Cloud Project

Module 5.1: AWS Monitoring

- CloudWatch
- Monitoring Metrics
- Alerts & Notifications

Module 5.2: Cloud Operations

- Cost Optimization Basics
- Security Best Practices
- Operational Excellence

Practice Session

- Monitor and Optimize Application

Deliverable

- Full Stack Application Deployed on AWS with Monitoring & Agile Project Simulation
-

Week 12 : Soft Skills Programme (Refer Annexure : HARTFORD <> HCL-GUVI : Training curriculum for Soft Skills for the detailed curriculum)

WEEK 13: AI for Developers

DAY 1: AI-Powered Development with Gemini + Prompt Engineering

MODULE 1.1: AI-Assisted Backend Development with Gemini (GCP Ecosystem)

- Gemini 1.5 Pro / Flash capabilities in development
- AI-assisted.NET API generation
- Context-aware coding workflows
- Secure AI coding practices (data leakage, hallucination control)

Practice Session:

- Generate Claims Management REST API using AI
 - Refactor into layered architecture:
 - Controller
 - Service
 - Repository
-

MODULE 1.2: Prompt Engineering for Software Engineers

- structured prompting (role + constraints + output schema)
- API-first prompt design
- deterministic output generation
- iterative prompt refinement strategies

Practice Session:

- Design prompts to generate:
 - Claims API

- validation rules
 - DTO structures
-

MODULE 1.3: AI Debugging + Refactoring Workflow

- using AI as a debugging assistant (not code author)
- legacy system refactoring patterns
- improving maintainability using AI suggestions

Practice Session:

- Refactor monolithic service into clean microservice-style structure
-

MODULE 1.4: AI-Driven System Design

- converting requirements → API contracts using AI
- schema-first design approach
- validation + business rules generation

Practice Session:

- Design full Claims API system using AI prompts
-

DAILY DELIVERABLE

- Claims API (AI-assisted but engineered manually)
 - Prompt library for backend generation
 - Clean layered architecture implementation
-

DAY 2: Document Intelligence + RAG Foundations + Vector Databases

MODULE 2.1: Gemini Document AI (Vertex AI)

- multimodal AI (text + PDF + images)
- structured extraction using Gemini
- document understanding pipelines

Practice Session:

- Extract structured insurance claim data from PDF:
 - name
 - policy number
 - claim amount
 - incident date
-

MODULE 2.2: Introduction to Vector Databases (CORE ADDITION)

- embeddings concept (text → vectors)
- similarity search basics
- why traditional DBs fail for AI search
- vector DB use cases in enterprises
- Overview of:
 - Vertex AI Vector Search
 - Pinecone (conceptual reference)
 - FAISS (local dev concept)

Practice Session:

- Convert policy documents into embeddings (conceptual + implementation design)
 - Store and retrieve similar insurance policies
-

MODULE 2.3: RAG Architecture (Retrieval-Augmented Generation)

- retrieval layer + generation layer
- grounding LLM responses with data
- chunking + embedding strategy

Practice Session:

- Build RAG flow design for:
 - insurance policy Q&A system

MODULE 2.4: Backend Integration of Document AI + RAG

- AI service layer in .NET
- embedding retrieval flow
- structured JSON output handling

Practice Session:

- API: /query-policy
 - Flow:
 - query → vector search → Gemini response
-

DAILY DELIVERABLE

- Document extraction pipeline
 - Vector DB-based retrieval design
 - Basic RAG system architecture
-

DAY 3: Google ADK (Agent Development Kit) + Agent Engineering

MODULE 3.1: Introduction to Google ADK (REPLACEMENT FOR LANGCHAIN/LANGGRAPH)

- ADK architecture overview
 - agent definition in GCP ecosystem
 - tools, memory, and orchestration
 - difference from LangChain agents
-

MODULE 3.2: Building Agents with ADK

- agent structure (input → reasoning → tool → output)
- tool registration in ADK
- API-based agent execution

Practice Session:

- Build Insurance Policy Q&A Agent using ADK
-

MODULE 3.3: Tool Integration in ADK

- database tool integration
- calculator / business logic tools
- API tool chaining

Practice Session:

- Agent with:
 - policy lookup tool
 - premium calculator tool
-

MODULE 3.4: Memory + Context Handling in ADK

- session memory handling
- contextual reasoning
- retrieval-based memory concept (RAG link)

Practice Session:

- Build conversational insurance assistant with memory
-

DAILY DELIVERABLE

- ADK-based AI agent
 - tool-enabled reasoning system
 - memory-enabled assistant
-

DAY 4: Vertex AI Agent Systems + RAG + Multi-Agent Design (ADK-Based)

MODULE 4.1: Vertex AI Agent Deployment

- deploying ADK agents on Vertex AI
- API exposure of agents
- cloud-native AI services

Practice Session:

- Deploy Claims Status Agent
-

MODULE 4.2: RAG System Implementation (Deep Dive)

- embedding pipeline design
- retrieval ranking logic
- grounding responses in enterprise data

Practice Session:

- Build policy knowledge RAG system using vector DB
-

MODULE 4.3: Multi-Agent Systems using ADK

- task decomposition across agents
- orchestration design
- agent role specialization

Practice Session:

- Design:
 - validation agent
 - processing agent
-

MODULE 4.4: Agent Workflow Engineering

- chaining agents using ADK orchestration
- state passing between agents
- failure recovery flow

Practice Session:

- 2-agent claims processing system
-

DAILY DELIVERABLE

- deployed ADK agent system
 - RAG-based Q&A system
 - multi-agent workflow design
-

DAY 5: Production AI Systems + GCP Deployment + Capstone

MODULE 5.1: Production AI Engineering Principles

- prompt safety and hallucination control
- PII handling in AI systems
- cost optimization for Gemini calls
- monitoring AI responses

Practice Session:

- Add validation layer to AI outputs using schema rules
-

MODULE 5.2: Deployment on GCP Cloud Run

- containerizing AI backend services

- deploying .NET + ADK service
- authentication layer setup

Practice Session:

- Deploy Claims AI service to Cloud Run
-

MODULE 5.3: AI System Integration Architecture

- AI as microservice pattern
- synchronous vs async AI calls
- caching AI responses
- backend + AI coupling strategy

Practice Session:

- Integrate AI service into full backend system
-

MODULE 5.4: AI Capstone System (GCP + ADK + RAG + Vector DB)

Choose ONE:

Option A: Claims Intelligence System

- upload document
- Gemini extraction
- vector DB storage
- RAG-based retrieval
- claim response generation

Option B: Insurance Policy AI Assistant

- ADK agent-based chatbot
- vector DB knowledge retrieval
- memory-enabled conversations
- tool-based reasoning

Option C: Document Risk Analyzer

- document upload
- AI extraction + embedding storage

- anomaly detection via retrieval
 - risk scoring report
-

FINAL SESSION

- architecture presentation
 - ADK workflow explanation
 - RAG + vector DB design walkthrough
 - deployment demo on GCP
-

WEEK 13 FINAL DELIVERABLE

- ADK-based enterprise AI agent system
- full RAG pipeline with vector database
- Gemini-powered document intelligence system
- deployed AI microservice on GCP Cloud Run
- production-ready AI integration architecture

Week 14 : Capstone Project

- Build a .NET Core REST API to manage employee data with add, view, update, and delete operations.
- Secure the API using login and role-based access.
- Create a web interface with Angular and Tailwind to input employee details and view lists.
- Connect the frontend seamlessly with the backend API.
- Use SQL Server to store employee data and manage it through Entity Framework Core.
- Implement error handling with friendly messages.
- Add search, filter, and sorting features for employee records.

- Follow good coding practices including SOLID principles and clean code.
- Deploy the fullstack project to AWS cloud.
- Set up a CI/CD pipeline for automated deployment.